Hybrid PTR-PPO 알고리즘을 활용한 클라우드 GPU 자원 할당 최적화

Optimizing Cloud GPU Resource Allocation using Hybrid PTR-PPO Algorithm

요 약

최근 GPU는 개인뿐만 아니라 데이터 센터에서도 클라우드 서비스 형태로 제공되고 있다. 그러나 과거의 알고리즘은 트래픽 변화에 빠르게 적응하기 어렵다. 따라서 본 보고서는 LSTM 기반의 미래 수요 예측과 PTR(Prioritized Trajectory Replay) 기법을 결합한 선제적(Proactive) 오토스케일링 알고리즘인 Hybrid PTR-PPO를 제안한다. 2025년 Alibaba Cluster Trace를 활용한 실험 결과, Hybrid PTR-PPO를 이용하여 기존 모델 대비 정책 학습 반복 횟수가 감소하였다.

1. 서 론

1.1 연구 배경

최근 인공지능(AI)과 머신러닝(ML) 기술이 급격히 발전함에 따라, 이를 처리하기 위한 연산 자원인 GPU(그래픽 처리 장치)의 수요가 폭발적으로 증가하고 있다. 이에 따라 기업과 연구소는 물리적인 GPU 서버를 직접 구축하기보다, 클라우드 데이터 센터에서 제공하는 가상 GPU 인스턴스를 임대하여 사용하는 추세다. 클라우드 환경은 필요에 따라 자원을 유연하게 확장 및 축소할 수 있다는 장점이 있으나, 이는 데이터 센터 운영자에게 효율적인 자원 분배라는 어려운 과제를 부여한다.

1.2 문제 정의

데이터 센터 운영의 핵심은 트래픽 수요 변동에 맞춰 적절한 수의 GPU를 할당하는 것이다. 만약 GPU를 과잉 할당할 경우 불필요한 운영 비용과 전력 낭비가 발생하며, 반대로 부족하게 할당할 경우 서비스 지연이나 중단이 발생하여 사용자 경험과 서비스 신뢰도에 부정적인 영향을 미친다. 특히, 실제 워크로드는 특정 시간대에 수요가 급증하는 현상이 빈번하여 예측이 매우 어렵다. 기존의 정적 임계치(Static Threshold) 기반이나 단순 휴리스틱 스케줄링 기법은 이러한 동적 변화에 유연하게 대처하지 못한다는 한계가 있다.

1.3 기존 연구의 한계 및 동기

이러한 문제를 해결하기 위해 최근 강화학습(Reinforcement Learning, RL)을 활용한 자원 관리 연구가 활발히 진행되고 있다. 그중 PPO(Proximal Policy Optimization)는 안정적인 학습 성능으로 주목받고 있으나, 두 가지 측면에서 한계를 보인다. 첫째, 현재 상태만을 관측하여 행동을 결정하는 방식이기 때문에, 트래픽이 급증한 이후에야 뒤늦게 자원을 늘리는 지연 문제가 발생한다. 둘째, PPO는 한 번 사용한 데이터를 폐기하는 'On-policy' 특성상 데이터 효율성(Sample Efficiency)이 낮아, 학습에 많은 시간이 소요된다.

1.4 본 연구의 제안 및 기여

본 연구에서는 상기한 한계점을 극복하기 위해, LSTM(Long Short-Term Memory) 기반의 수요 예측과 PTR(Prioritized Trajectory Replay) 기법을 결합한 Hybrid PTR-PPO 알고리즘을 제안한다. 제안하는 모델은 LSTM을 통해 미래의 GPU 수요를 미리 예측하여 상태 벡터에 반영함으로써, 에이전트가 트래픽 변동을 사전에 인지하고 선제적으로 대응하도록 유도한다. 또한, PTR 기법을 도입하여 과거의 중요한 경험을 재사용함으로써 제한된

데이터 환경에서도 높은 학습 효율성을 달성한다. 본 연구는 2025년 Alibaba Cluster Trace 데이터[2]를 활용하여 제안 모델의 성능을 검증하였으며, 실험 결과 기존 PPO 대비 빠른 수렴 속도와 안정적인 자원 할당 능력을 입증하였다.

2. 관련 연구

2.1 클라우드 자원 스케줄링 기법

클라우드 컴퓨팅 환경에서의 자원 스케줄링은 다양한 요구사항을 만족시키면서 시스템의 효율성을 극대화하는 것을 목표로 한다. 본 절에서는 전통적인 휴리스틱 기반 기법과 최근 연구되고 있는 강화학습 기반 기법의 특징 및 한계를 기술한다.

2.1.1 전통적인 휴리스틱 및 정적 할당 기법

초기의 클라우드 데이터 센터에서는 자원 관리를 위해 주로 정적 할당이나 휴리스틱(Heuristic) 알고리즘을 사용하였다.[1] 대표적 예로 선입선출(First-In-First-Out), SJF(Shortest Job First) 같은 알고리즘이 있다. 또한, CPU나 GPU 사용률에 대한 특정 임계값(Threshold)을 설정하여, 이를 초과하거나 미달할 경우 자원을 확장 및 축소하는 규칙 기반(Rule-based) 오토스케일링 방식이 널리 적용되어 왔다.

2.1.2 강화학습 기반 스케줄링 기법

정적 기법의 한계를 극복하기 위해, 환경과의 상호작용을 통해 동적으로 최적의 정책을 학습하는 강화학습(Reinforcement Learning)을 스케줄링에 적용하는 연구가 활발히 진행되고 있다.

1) 가치 기반(Value-based) 접근: Q-Learning

Q-Learning은 특정 상태에서 행동을 취했을 때 기대되는 보상의 총합인 Q-Value를 학습하는 방식이다. 초기 연구들에서는 Q-Table을 이용하여 최적의 스케줄링 정책을 학습하려 시도하였으나 , 상태 공간과 행동 공간이 커질수록 Q-Table의 크기가 기하급수적으로 증가하여 고차원 문제에 적용하기 어렵다는 '차원의 저주' 문제가 발생한다.

2) 정책 기반(Policy-based) 접근: A2C 및 PPO

이러한 한계를 해결하기 위해 신경망을 활용한 정책 최적화 기법들이 도입되었다.

A2C(Advantage Actor-Critic)는 정책을 결정하는 Actor와 가치를 평가하는 Critic 네트워크를 동시에 학습시켜 안정성을 높이고 계산량을 줄였다 . 하지만 A2C 역시 하이퍼파라미터에 민감하고 학습 과정이 불안정할 수 있다는 단점이 있다. 이에 OpenAI는 PPO(Proximal Policy Optimization)를 제안하였다. PPO는 새로운 정책이 기존 정책에서 너무 크게 벗어나지 않도록 업데이트 비율을 제한하는 클리핑(Clipping) 기법을 적용한다. 이를 통해 학습의 안정성을 획기적으로 개선하였으며, 이산적(Discrete) 공간과 연속적(Continuous) 공간 모두에서 우수한 성능을 보여 자원 스케줄링과 같은 복잡한 제어 문제에 널리 활용되고 있다.

2.2 PPO (Proximal Policy Optimization)

2.2.1 PPO의 개념 및 도입 배경

PPO는 OpenAI가 제안한 정책 최적화 알고리즘으로, 기존의 정책 그라디언트(Policy Gradient) 방식이 학습률(Learning Rate) 등 하이퍼파라미터에 민감하여 학습이 불안정해지는 문제를 해결하기 위해 고안되었다. PPO의 핵심 아이디어는 새로운 정책이 이전 정책과 비교했을 때 너무 급격하게 변화하지 않도록 제한(Constraint)을 두는 것이다. 이를 통해 학습의 안정성을 확보하면서도 구현이 간단하여 다양한 강화학습 문제에서 널리 사용되고 있다.

2.2.2 목적 함수와 클리핑(Clipping) 기법

PPO는 정책 업데이트의 폭을 제한하기 위해 클리핑(Clipping) 기법을 도입한 목적 함수(Objective Function)를 사용한다. PPO의 목적 함수는 다음과 같이 정의된다.

$$L^{CLIP}(\theta) = \min[r_t(\theta) \cdot A_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon) \cdot A_t]$$

(단, $r_t(\theta) = \dfrac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$)

여기서 $r_t(\theta)$는 이전 정책에 대한 새로운 정책의 확률 비율(Probability Ratio)을 의미한다. A_t는 어드밴티지 함수(Advantage Function)로, 현재 상태에서 취한 행동이 평균보다 얼마나 더 좋은지를 나타내는 지표이다. ϵ은 클리핑 범위를 결정하는 하이퍼파라미터로, 통상적으로 0.1 또는 0.2의 값을 사용한다.

이 수식의 의미는 $r_t(\theta)$가 1-ϵ과 1+ϵ의 범위를 벗어날 경우, 목적 함수의 값을 잘라내어(Clip) 정책이 과도하게 업데이트되는 것을 방지하는 것이다. 즉, 정책 변화의 상한선과 하한선을 두어 보수적이고 안정적인 학습을 유도한다.

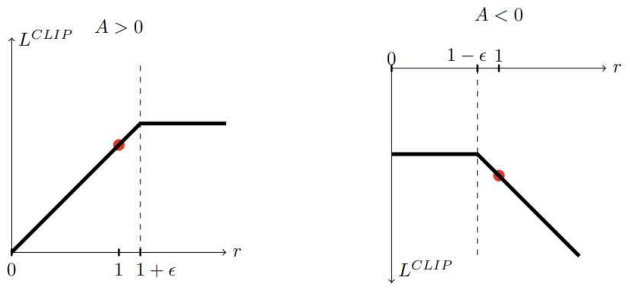

그림 1: 클리핑의 예시를 보여주는 그래프

2.2.3 PPO의 장점과 한계

PPO는 A2C와 같은 기존 알고리즘 대비 학습 안정성이 뛰어나며, 이산적 행동 공간과 연속적 행동 공간 모두에 적용하기 용이하다는 범용성을 가진다. 이러한 특성은 자원 할당과 같이 안정적인 정책 수립이 중요한 문제에 적합하다.

그러나 PPO는 'On-policy' 알고리즘의 특성상, 현재 정책으로 수집한 데이터만을 학습에 사용하고 폐기해야 한다. 한 번 사용된 데이터는 재사용되지 않기 때문에 샘플 효율성이 낮다는 한계가 존재한다. 이는 데이터를 무한정 생성하기 어려운 실제 데이터 센터 환경에서 학습 속도를 저하시키는 요인이 되며, 본 연구에서 이를 개선하기 위해 PTR(Prioritized Trajectory Replay) 기법을 도입하는 주된 동기가 된다.

2.3 PTR-PPO (Prioritized Trajectory Replay)

2.3.1 개념 및 도입 배경

PPO는 학습의 안정성이 뛰어나지만, 현재 정책(Current Policy)으로 수집한 데이터만을 학습에 사용하고 폐기하는 'On-policy' 특성을 가진다. 이로 인해 한 번 수집된 데이터가 재사용되지 못하여 데이터 효율성(Sample Efficiency)이 낮다는 단점이 존재한다. 데이터 수집 비용이 높거나 시뮬레이션 시간이 오래 걸리는 환경에서는 이러한 비효율성이 학습 속도를 저하시키는 주된 원인이 된다.

이를 해결하기 위해 제안된 PTR-PPO는 PPO의 안정적인 클리핑(Clipping) 구조를 유지하면서, 과거의 경험을 저장하고 재사용하는 리플레이 버퍼(Replay Buffer) 개념을 도입한 알고리즘이다.[3]

2.3.2 핵심 메커니즘: 우선순위 기반 리플레이

PTR-PPO의 가장 큰 특징은 단순히 과거 데이터를 무작위로 재사용하는 것이 아니라, 학습에 도움이 되는 '중요한 경험'을 선별한다는 점이다. 구체적인 동작 원리는 다음과 같다.

1. 트라젝토리 저장: 에이전트가 환경과 상호작용하며 얻은 상태, 행동, 보상 등의 롤아웃(Rollout) 데이터를 리플레이 버퍼에 저장한다.

2. 우선순위(Priority) 산정: 저장된 트라젝토리 중 어드밴티지(Advantage) 절댓값이 큰 데이터, 즉 현재 정책이 예측하지 못했거나 학습할 여지가 많은 데이터를 우선적으로 샘플링한다.

3. 학습(Update): 현재 정책으로 수집한 On-policy 데이터와 버퍼에서 추출한 Off-policy 데이터를 결합하여 정책을 업데이트한다.

이러한 메커니즘을 이용해 PTR-PPO는 제한된 데이터 환경에서도 높은 샘플 효율성을 달성할 수 있다. 또한, PPO에 대비해 적은 반복으로 학습을 진행할 수 있다.

3. 제안하는 방법

3.1 문제 정의

본 연구에서는 클라우드 데이터 센터의 GPU 오토스케일링 문제를 에이전트가 환경과 상호작용하며 보상을 최대화하는 방식으로 모델링한다.

3.1.1 상태 공간 (State Space)

에이전트가 현재 시점 t에서 관측하는 상태 S_t는 과거의 GPU 수요 패턴과 미래의 예측 정보를 모두 포함하도록 설계되었다. 기존 연구들이 현재 시점의 자원 상태만을 고려한 것과 달리, 본 연구에서는 LSTM 모델을 통해 예측된 미래 수요를 상태 벡터에 포함하여 선제적 대응이 가능하도록 하였다.

$$S_t = [d_{t-w+1}, \dots, d_t, \hat{d}_{t+1}]$$

d_t: 시점 t에서의 실제 GPU 수요 (Normalized)

w: 과거 데이터를 참조하는 윈도우 크기 (Window Size)

$\hat{d}_{t+1}$: LSTM 모델이 예측한 $t+1$ 시점의 GPU 수요

3.1.2 행동 공간 (Action Space)

행동 A_t는 에이전트가 선택할 수 있는 GPU 용량 수준(Capacity Level)을 의미한다. 본 연구에서는 행동 공간을 이산적(Discrete)인 단계로 정의하여, 에이전트가 전체 자원 한도 내에서 10단계의 용량 수준 중 하나를 선택하

도록 하였다.

$$A_t \in \{L_1, L_2, \dots, L_{10}\}$$

여기서 L_k는 전체 최대 용량의 $k \times 10\%$에 해당하는 자원 할당량을 나타낸다. (예: 10%, 20%, ... ,100%)

3.1.3 보상 함수 (Reward Function, R)

보상 R_t는 에이전트의 행동이 목표에 얼마나 부합하는지를 평가하는 지표이다. 본 연구의 목표는 자원 낭비(Idle)를 최소화하면서도, 서비스 신뢰도에 치명적인 자원 부족(Shortage)을 방지하는 것이다. 따라서 자원 부족에 더 큰 페널티 가중치를 부여한 보상 함수를 정의하였다.

Shortage$_t$ = max($d_t - c_t$, 0): 수요가 할당량(c_t)을 초과한 양

Idle$_t$ = : max($c_t - d_t$, 0): 할당량이 수요를 초과하여 낭비된 양

α, β: 페널티 가중치 (본 연구에서는 α=2.0, β=1.0 설정)

3.2 Hybrid 아키텍처 및 시스템 구성

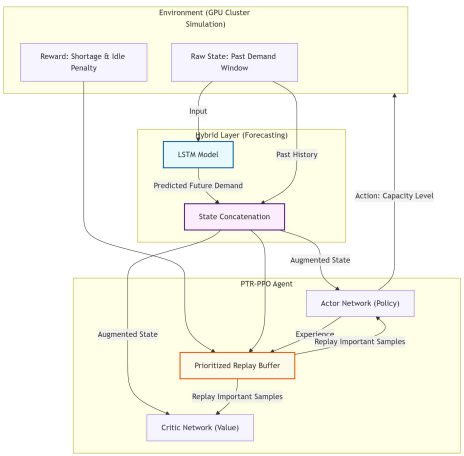

그림 2: Hybrid PTR-PPO 아키텍처

본 연구에서 제안하는 Hybrid PTR-PPO 시스템의 전체적인 아키텍처는 [그림 2]와 같다. 이 시스템은 크게 세 가지 주요 계층인 환경(Environment), 하이브리드 계층(Hybrid Layer), 그리고 PTR-PPO 에이전트(Agent)로 구성된다.

3.2.1 환경 및 상태 관측 (Environment & Observation)

가장 상단의 환경 계층은 Alibaba GPU Cluster Trace 데이터를 기반으로 시뮬레이션 된 가상 환경이다. 환경은 에이전트의 행동(Capacity Control)에 따라 변화하며, 각 시점 t마다 두 가지 정보를 생성한다.

Raw State (과거 수요 윈도우): 최근 w개의 시점 동안의 실제 GPU 수요 데이터 $(d_{t-w+1}, \dots, d_t)$

Reward: 에이전트의 이전 행동에 대한 평가 지표로, 자원 부족(Shortage)과 낭비(Idle)에 대한 페널티의 가중 합으로 계산된다.

3.2.2 하이브리드 예측 모듈 (Hybrid Layer)

기존 강화학습 모델이 과거 데이터(Raw State)에만 의존하여 반응형(Reactive)으로 동작하는 한계를 극복하기 위해, 중간 계층에 하이브리드 예측 모듈을 도입하였다.

LSTM 모델: 시계열 데이터 처리에 특화된 LSTM 신경망은 Raw State를 입력받아 미래 시점($t+1$)의 GPU 수요 ($\hat{d}_{t+1}$)를 예측한다.

상태 결합 (State Concatenation): 환경에서 제공된 과거 수요 데이터와 LSTM이 예측한 미래 수요 데이터를 결합하여 확장된 상태 벡터(Augmented State)를 생성한다. 이 확장된 상태는 에이전트가 현재 상황뿐만 아니라 미래의 트래픽 변동까지 고려하여 의사결정을 내릴 수 있도록 돕는다.

3.2.3 PTR-PPO 에이전트 (Agent with Prioritized Replay)

하단의 에이전트 계층은 확장된 상태를 입력받아 최적의 정책을 학습하고 행동을 결정한다.

Actor & Critic 네트워크: 확장된 상태를 기반으로 각각 행동(GPU 용량 수준)을 결정하고, 상태 가치를 평가한다.

우선순위 리플레이 버퍼 (Prioritized Replay Buffer): 에이전트가 경험한 트라젝토리(Trajectory)를 저장한다. 이때 Time-Difference Error가 큰, 즉 학습에 중요한 경험에 높은 우선순위를 부여하여 저장하고, 학습 시 이를 우선적으로 샘플링하여 재사용(Replay)한다. 이는 데이터가 제한적인 환경에서 샘플 효율성을 극대화하는 핵심 역할을 수행한다.

3.3 학습 안정화 기법

강화학습 에이전트가 복잡한 클라우드 환경에서 안정적으로 수렴하고, 실제 서비스에 적용 가능한 정책을 학습하기 위해 다음과 같은 안정화 기법들을 적용하였다.

3.3.1 무작위 초기화 (Random Reset Strategy)

시계열 데이터를 기반으로 강화학습을 수행할 때, 매 에피소드를 데이터의 시작점(Time step 0)에서만 시작할 경우 에이전트가 특정 시점의 패턴을 단순 암기하는 과적합(Overfitting) 문제가 발생할 수 있다. 이를 방지하고 일반화 성능을 높이기 위해, 본 연구에서는 에피소드 시작 시 전체 트레이스 데이터 구간 내 임의의 시점을 시작점(t_{start})으로 설정하는 Random Reset 기법을 적용하였다.

$$t_{\text{start}} \sim \text{Uniform}(0,\ T - \text{Horizon})$$

이를 통해 에이전트는 특정 시간대에 국한되지 않고 다양한 트래픽 패턴(Low traffic, Burst traffic 등)을 골고루 학습하여 변화에 강한 정책을 수립할 수 있다.

3.3.2 비대칭 보상 설계

클라우드 자원 관리에서 자원 낭비(Idle)는 비용 문제에 그치지만, 자원 부족(Shortage)은 서비스 품질 저하로 직결되는 치명적인 문제이다. 따라서 본 연구에서는 두 가지 페널티에 서로 다른 가중치를 부여하는 비대칭 보상 함수를 설계하였다. 자원 부족에 대한 페널티 가중치($\alpha = 2.0$)를 자원 낭비에 대한 가중치($\beta = 1.0$)보다 높게 설정함으로써, 에이전트가 불확실한 상황에서는 다소 보수적으로 자원을 확보하는 방향으로 학습하도록 유도하여 서비스 안정성을 최우선으로 보장하였다.

4. 실험 진행 및 결과

4.1 실험 환경

4.1.1 데이터셋 및 전처리

본 연구에서는 실제 클라우드 데이터 센터의 워크로드 패턴을 반영하기 위해 Alibaba Cluster Trace v2025 데이터를 사용하였다. 이 데이터셋은 알리바바의 대규모 GPU 클러스터에서 수집된 실제 운영 로그로, 작업(Job)의 생성 시간(creation_time), 종료 시간(deletion_time), 그리고 GPU 요청량(gpu_request) 등의 정보를 포함하고 있다.

데이터 전처리는 다음과 같은 과정을 통해 진행하였다.

1. 필터링: 전체 트레이스 중 GPU 자원을 요청한 (gpu_request > 0) 작업만을 추출하였다.
2. 시계열 변환 (Time-series Aggregation): 불규칙한 이벤트 로그를 정형화된 시계열 데이터로 변환하기 위해, 전체 시간을 일정 간격(Bin size)으로 나누고 각 구간에서 활성화된(Active) GPU 인스턴스의 총합을 계산하여 수요량(Demand)을 산출하였다.
3. 정규화 (Normalization): 학습 안정성을 높이기 위해 전체 수요 데이터를 최대 수요량(Max Demand)으로 나누어 [0, 1] 범위로 정규화하였다.

4.1.2 시뮬레이션 환경 설정

에이전트의 학습을 위해 OpenAI Gym 인터페이스를 따르는 커스텀 시뮬레이션 환경(GpuCapacityEnv)을 구축하였다.

상태 정의: 에이전트는 과거 w개의 시점(Window Size) 동안의 수요 패턴과 LSTM 예측값을 포함한 $w+1$ 차원의 벡터를 상태로 관측한다. 본 실험에서는 $w = 10$로 설정하였다.

행동 정의: 에이전트는 전체 최대 용량의 10% 단위로 구분된 10단계의 이산적인(Discrete) 용량 수준 중 하나를 선택한다.

보상 설정: 서비스 안정성을 최우선으로 고려하여 자원 부족(Shortage)에 대한 페널티 가중치(α)를 2.0, 자원 낭비(Idle)에 대한 가중치(β)를 1.0으로 설정하였다.

4.1.3 하이퍼파라미터 설정

제안 모델과 베이스라인 모델의 공정한 비교를 위해 공통적인 PPO 하이퍼파라미터는 동일하게 설정하였다. PTR-PPO에 특화된 리플레이 버퍼 관련 파라미터는 사전 실험을 통해 최적화된 값을 사용하였다. 주요 파라미터 설정은 [표 1]과 같다.

구분	parameter	value
Common	Discount Factor (γ)	0.99
	GAE Parameter (λ)	0.95
	Learning Rate	3e-4
	Clip Range (ϵ)	0.2
	Horizon	128
PTR-PPO	Buffer Capacity	16
	Replay Count (K)	2
	Priority Exponent (α)	0.4
	IW Clip	1.3

표 1: 실험에 사용된 주요 하이퍼파라미터

parameter	비고
Discount Factor (γ)	미래 보상 할인율
GAE Parameter (λ)	Advantage 추정용
Learning Rate	Adam Optimizer
Clip Range (ϵ)	PPO Clipping
Horizon	1회 Rollout 길이
Buffer Capacity	Trajectory 단위
Replay Count (K)	업데이트 당 재사용 횟수
Priority Exponent (α)	우선순위 반영 강도
IW Clip	Importance Weight 제한

표 2: 파라미터에 대한 설명

[표 2]는 설정한 파라미터들에 대한 설명이다.

4.2 베이스라인 비교

본 절에서는 미래 수요 예측(Forecasting) 정보 없이 과거의 상태(Raw State)만을 이용하여 학습한 베이스라인 모델인 PPO (Base)와 PTR-PPO (Base)의 성능을 비교 분석한다.

4.2.1 학습 수렴 속도

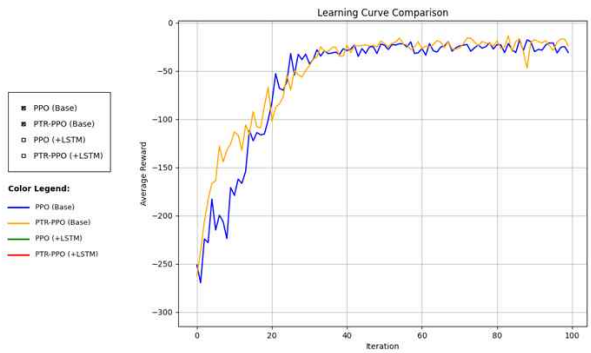

그림 3: PPO와 PTR-PPO 베이스라인 모델 학습 곡선

[그림 3]을 보면 PPO와 PTR-PPO 두 모델 모두 우상향하는 모습을 보여주고 있다. 그 중 PTR-PPO 모델의 경우 초반 구간에서 더 빠르게 학습이 진행되었다는 사실을 볼 수 있다. 즉 더 적은 반복(iteration)으로 최선의 정책을 찾아가고 있음을 확인할 수 있다.

4.2.2 Forecasting 없는 PTR-PPO의 한계

PTR-PPO (Base) 모델은 리플레이 버퍼를 도입하여 데이터 재사용성을 높였음에도 불구하고, LSTM 예측 정보의 부재로 인해 성능 향상에 한계를 보였다. 과거의 중요 경험을 재학습하더라도, 상태 정보(State)에 미래 수요에 대한 단서가 포함되어 있지 않아 트래픽 급증(Burst) 시점을 미리 파악하고 대비하는 정책을 학습하기 어려웠다. 따라서 단순한 PTR 기법의 적용만으로는 시계열 특성이 강한 워크로드 환경에서 학습 속도를 획기적으로 단축하는 데 한계가 있었으며, 이는 효율적인 자원 할당을 위해 미래 예측 정보(Forecasting)의 결합이 필수적임을 시사한다.

4.3 제안 모델 성능

4.3.1 LSTM 도입에 따른 초기 학습 가속

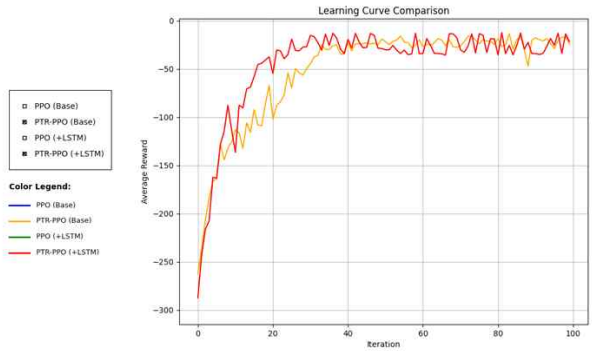

그림 4: PTR-PPO와 PTR-PPO+LSTM Hybrid 모델의 학습 곡선

실험 결과, LSTM 예측값을 상태 벡터에 포함한 Hybrid 모델과 그렇지 않은 베이스라인 모델(PTR-PPO, PTR-PPO+LSTM)을 비교했을 때 베이스라인 모델군 대비 Hybrid 모델이 더 빠른 초기 학습 속도를 기록하였다. 이는 [그림 4]의 학습 곡선을 비교해보면 쉽게 발견할 수 있다.

4.3.2 최종 결과

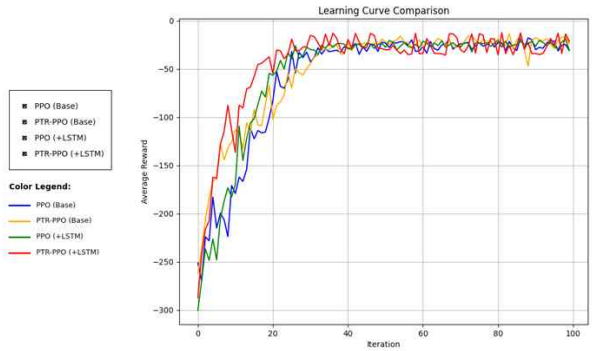

그림 5 베이스라인과 LSTM을 포함한 Hybrid 모델들을 비교한 학습 곡선

최종적으로 제안하는 Hybrid PTR-PPO 모델(빨간색 실선)은 모든 비교군 중에서 가장 적은 반복 횟수

(iteration)만으로 최적의 성능에 도달하였다. 이는 [그림 5] 에서 확인할 수 있다.

PPO 대비 향상: 단순 Hybrid PPO와 비교했을 때, Hybrid PTR-PPO는 우선순위 리플레이(Prioritized Replay) 기법을 통해 중요한 경험을 집중적으로 재학습함으로써 더 적은 샘플 수로도 빠르게 목표 성능에 수렴하였다.

시너지 효과: 결과적으로 LSTM의 '초기 가속 효과'와 PTR의 '높은 데이터 효율성'이 결합되어, 기존 PPO (Base) 모델 대비 빠른 속도로 수렴하는 결과를 달성하였다. 이는 데이터 수집 비용이 높거나 빠른 학습이 요구되는 실제 데이터 센터 환경에서 매우 중요한 이점이다.

5. 결론

5.1 연구 요약

본 연구에서는 클라우드 데이터 센터의 GPU 자원 효율성을 극대화하기 위해, 기존 반응형(Reactive) 스케일링의 한계를 극복하는 Hybrid PTR-PPO 알고리즘을 제안하였다. 제안하는 모델은 LSTM 신경망을 통해 미래의 워크로드 수요를 예측하여 선제적(Proactive) 대응이 가능하도록 설계되었으며, 우선순위 기반 트라젝토리 리플레이(PTR) 기법을 도입하여 제한된 데이터 환경에서도 높은 샘플 효율성을 달성하였다.

5.2 주요 성과 및 의의

2025년 Alibaba Cluster Trace 데이터를 기반으로 한 실험 결과, 제안 모델은 학습 속도의 개선을 이루었다. LSTM의 예측 정보를 상태(State)에 통합함으로써, 에이전트가 트래픽 변화를 사전에 인지하여 기존 PPO 대비 훨씬 빠른 속도로 최적 정책에 수렴하였다. 또한, 서비스 안정성 확보에 기여할 수 있다. 자원 부족(Shortage)에 높은 가중치를 둔 비대칭 보상 설계를 통해, 급격한 트래픽 변동 상황에서도 안정적인 자원 할당을 꾀할 수 있다. 본 연구는 클라우드 데이터의 특성을 반영한 시뮬레이션 환경에서 강화학습 기반 오토스케일링의 효율을 올렸다는 점에서 의의가 있다.

5.3 보완점

본 제안 기법은 우수한 성능을 보였으나, 리플레이 버퍼 유지와 우선순위 계산으로 인해 기존 PPO 대비 메모리 사용량과 연산 복잡도가 다소 증가한다는 한계가 있다. 향후 연구에서 효율적인 버퍼 관리 기법과 메모리 최적화를 적용하여 모델 경량화가 이루어진다면 실제 데이터 센터에서 적용할 수 있을 것으로 기대된다. 또한, 적절한 튜닝을 통해서 NPU(Neural Processing Unit)와 같은 다른 자원에도 대상을 확장할 수 있을 것으로 기대된다.

참 고 문 헌

[1] Nasim Soltani, "Heuristic Algorithms for Task Scheduling in Cloud Computing: A Survey", 2017

[2] https://github.com/alibaba/clusterdata/tree/master/cluster-trace-gpu-v2025

[3] Xingxing Liang, "Proximal Policy Optimization with Prioritized Trajectory Replay", 2021